PROGETTO
SVILUPPO DELLE APPLICAZIONI SOFTWARE

—

Salvatore Pierri

ANALISI DELLE STORIE E COMBINAZIONE

Analizziamo le storie degli attori per comprendere le loro esigenze, conduciamo una ricerca approfondita per ciascuna storia e successivamente integriamo i risultati, al fine di avviare la creazione del nostro Use Case dettagliato.

Intervista a Robert, il Proprietario e Organizzatore

La gestione del personale è una parte fondamentale del mio lavoro. Avere la squadra giusta influenza di molto il successo del lavoro. Mi confronto regolarmente con gli organizzatori per avere un'idea dei ruoli necessari e se abbiamo il personale necessario. Se si tratta di un evento formale, come ad esempio un matrimonio, avremo bisogno di più camerieri e sommelier in sala; mentre per quelli più informali, come i buffet dei meeting aziendali, dovremo concentrarci maggiormente sulla logistica e sulle pulizie. Anche in cucina occorre avere una corretta suddivisione dei ruoli in base al tipo di evento. Per un compleanno avrò bisogno di un pastry chef che si occupi delle torte, per un buffet di un *garde manger* che gestisca le preparazioni fredde. E così via. Poi, ovviamente, ci sono mansioni che devono sempre essere svolte, indipendentemente dal servizio. I piatti non si lavano da soli...

Il nostro personale è diviso in due categorie: permanente e occasionale. Considero sempre prima il personale permanente cercando di richiamare i lavoratori occasionali solo in caso di necessità. Ad esempio, a maggio veniamo spesso chiamati per l'organizzazione di prime comunioni e cresime anche in parallelo in luoghi diversi. I lavoratori occasionali lavorano tipicamente per qualche mese, oppure vengono richiamato per eventi specifici.

Per eventi specifici di grosse dimensioni, gli organizzatori possono richiedere con il dovuto anticipo dei ruoli senza specificare una persona e io cerco di trovare qualcuno per ricoprire tali ruoli. Ormai ho un elenco di persone che posso contattare via e-mail o whatsapp. Tengo traccia di chi ho contattato e gli do sempre una scadenza entro cui rispondere in modo da avere tempo per contattare qualcun altro in caso di rifiuto. Alla fine aggiungo il nuovo elemento alla squadra.

Ogni tanto ricevo delle segnalazioni per nuovo personale. In questo caso mi segno il nome e il contatto, completando le informazioni al primo ingaggio. Infatti, per fare il contratto ho bisogno anche dell'indirizzo e codice fiscale della persona. Se vedo dallo storico dei nostri eventi che alcune persone sono state richiamate regolarmente mi consulto con gli event manager per considerare la possibilità di offrire un posto permanente.

Quando ricevo richieste per periodi di ferie, devo decidere se possono essere accettate o meno, in base alla disponibilità residua di giorni di ferie del lavoratore e gli eventi complessivi sul tabellone dei turni, e se accettate devo aggiornare opportunamente l'informazione relativa al personale. Preferisco dipendenti che non mi lasciano nei guai nei periodi di maggior lavoro.

Intervista a Raffaele, un Organizzatore che supporta Robert quando necessario.

Organizzare il personale per un evento è come comporre un'orchestra: ogni persona deve essere al posto giusto per garantire un servizio perfetto. Il mio lavoro inizia molto prima dell'evento, raccogliendo le richieste del cliente e valutando tutte le esigenze logistiche. Un dettaglio può fare la differenza: il numero di ospiti, la location, il tipo di servizio richiesto, tutto incide sulla scelta delle persone che lavoreranno all'evento.

La prima fase del mio lavoro è collaborare con lo Chef per definire il numero di persone necessarie in cucina e in sala. Lui ha una visione più dettagliata delle necessità operative, mentre io devo considerare la fluidità del servizio e il comfort degli ospiti. Se il cliente desidera un servizio di alto livello, potremmo aver bisogno di più camerieri per mantenere un certo standard; se la location è difficile da raggiungere, potremmo dover trovare soluzioni per trasportare il personale. Tutte queste informazioni sono salvate nella scheda riepilogativa dell'evento.

Una volta che abbiamo stabilito la scheda riepilogativa, controllo le disponibilità del personale. Se alcuni ruoli rimangono scoperti contatto i nostri collaboratori occasionali dall'elenco. Solo gli organizzatori hanno accesso a questo elenco per la privacy dei dati. Ogni tanto capita che qualcuno non è più disponibile e lo cancello dall'elenco, oppure aggiorno i dettagli di altri, ad esempio cambiando il numero di telefono.

Assunzioni permanenti sono gestite direttamente dal proprietario ma ci contatta regolarmente sulle necessità e su chi ha lavorato meglio. Per questo dopo ogni evento mi faccio delle note.

Informazioni generali

Nome caso d'uso: Gestione Personale Catering
Portata: Sistema
Livello: Obiettivo utente
Attore primario: Organizzatore, Chef
Parti Interessate: Organizzatore, Personale e Cliente
Pre-condizioni: L'attore deve essere autenticato come Organizzatore o Chef
Garanzie di successo o post-condizioni: Con la creazione e selezione dell'evento è data a disposizione dell'utente l'utilizzo di 2 liste che si occuperanno di gestire il personale e l'inserimento di esso nei vari eventi

Scenario principale di successo

#	Attore	Sistema
1	L'Organizzatore avvia la procedura di creazione evento inserendo i dati richiesti (Titolo, Numero Clienti, Tipo di evento e Distanza della Location).	Il Sistema crea l'evento e restituisce un riepilogo iniziale insieme a una prima lista di personale suggerito o disponibile.
2	**[Opzionale]** L'Organizzatore decide di aggiungere specifici membri dello staff all'evento:	L'Organizzatore inserisce l'identificativo (taxID) del membro dello staff. Il Sistema recupera il profilo dalla lista generale e lo associa all'evento corrente. Il passaggio può essere ripetuto per più membri.
3	**[Opzionale]** L'Organizzatore richiede la stampa o la visualizzazione del riepilogo finale dello staff confermato per l'evento specifico.	Il Sistema stampa l'elenco dello staff nel determinato evento

Estensione 1a/2a/2b/2c

#	Attore	Sistema
1a	L'Organizzatore seleziona un evento specifico dal sistema.	Il Sistema restituisce il riepilogo dei dettagli dell'evento e la lista attuale del personale assegnato.
2a	L'Organizzatore aggiunge un membro.	Il Sistema lo inserisce nella lista dell'evento e ne cambia lo stato in "occupato".
2b	L'Organizzatore aggiorna i dati di un membro dello staff.	Il Sistema conferma l'avvenuta modifica delle informazioni.
2c	L'Organizzatore rimuove un membro dall'evento.	Il Sistema lo esclude dalla lista dell'evento e ne ripristina lo stato a "libero".
3	**[Opzionale]** Indipendentemente dalla modifica effettuata, l'Organizzatore può richiedere la stampa o la visualizzazione del riepilogo aggiornato della lista staff per l'ID evento selezionato.	Il Sistema stampa l'elenco dello staff nel determinato evento

Eccezioni 1a/2a/2b/2c

#	Attore	Sistema
2a	L'Organizzatore aggiunge un membro.	Il Sistema prova ad inserire il personale, ma è già occupato, stampa un errore.
2b	L'Organizzatore modifica un membro.	Il Sistema prova a modificare il personale, ma il taxID non esiste, stampa un errore.
2c	L'Organizzatore rimuove un membro dall'evento.	Il Sistema prova ad eliminare il personale, ma non fa parte dell'evento, stampa un errore.

Estensione 1b/1c/1d/2

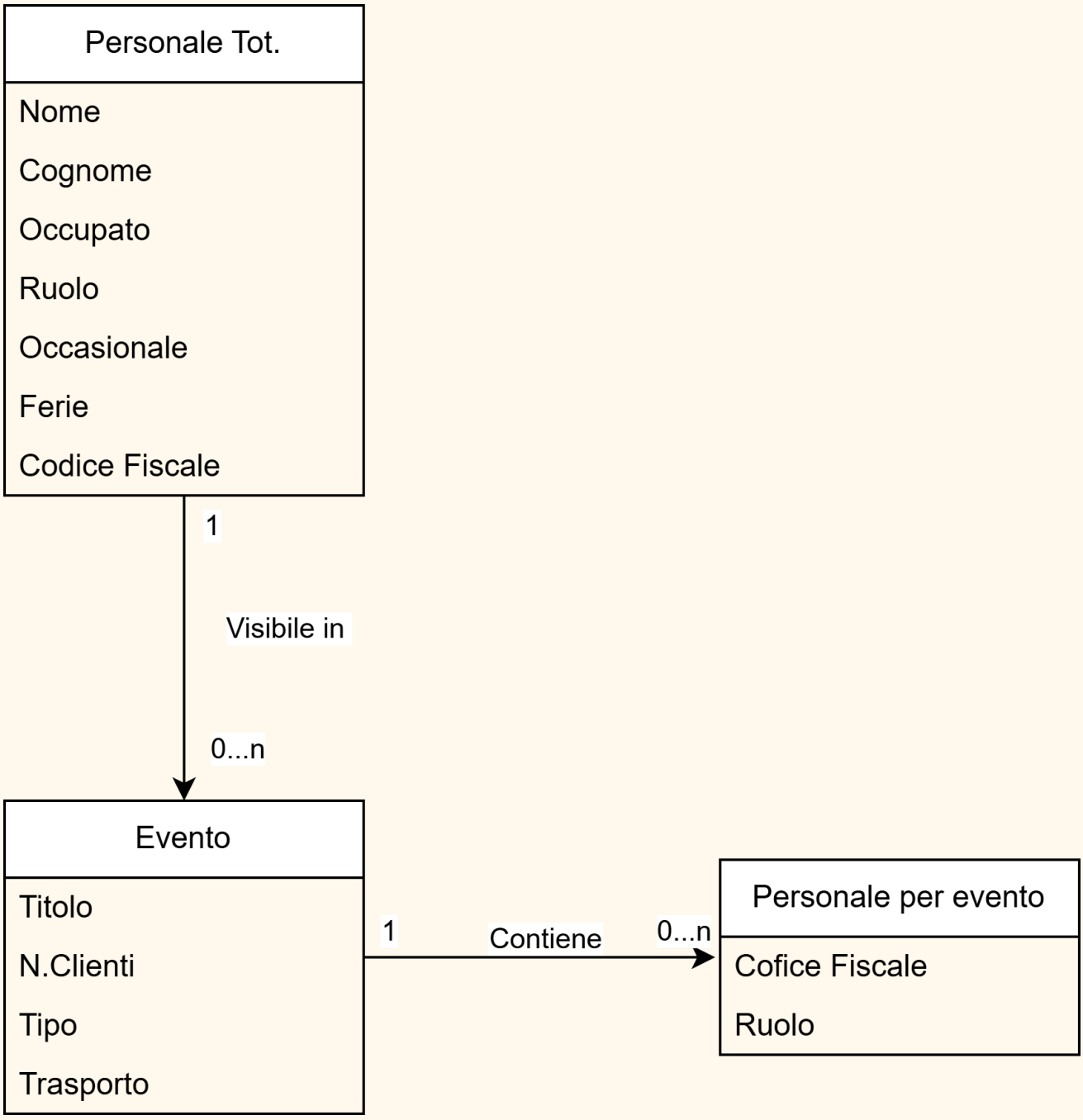
#	Attore	Sistema
1b	L'Organizzatore registra un nuovo membro inserendo dati completi.	Il sistema lo aggiunge alla lista generale.
1c	L'Organizzatore rimuove definitivamente un membro dalla lista generale.	Il sistema lo rimuove tramite il suo `taxID`.
1d	L'Organizzatore inserisce il numero di ferie richieste da un membro del personale.	Il sistema lo modifica tramite il suo `taxID` ed il numero di ferie.
2	**[Opzionale]** L'utente può richiedere in qualsiasi momento la stampa del riepilogo della lista staff generale.	Il Sistema stampa l'elenco dello staff nel determinato evento

Eccezioni 1b/1c/1d/2

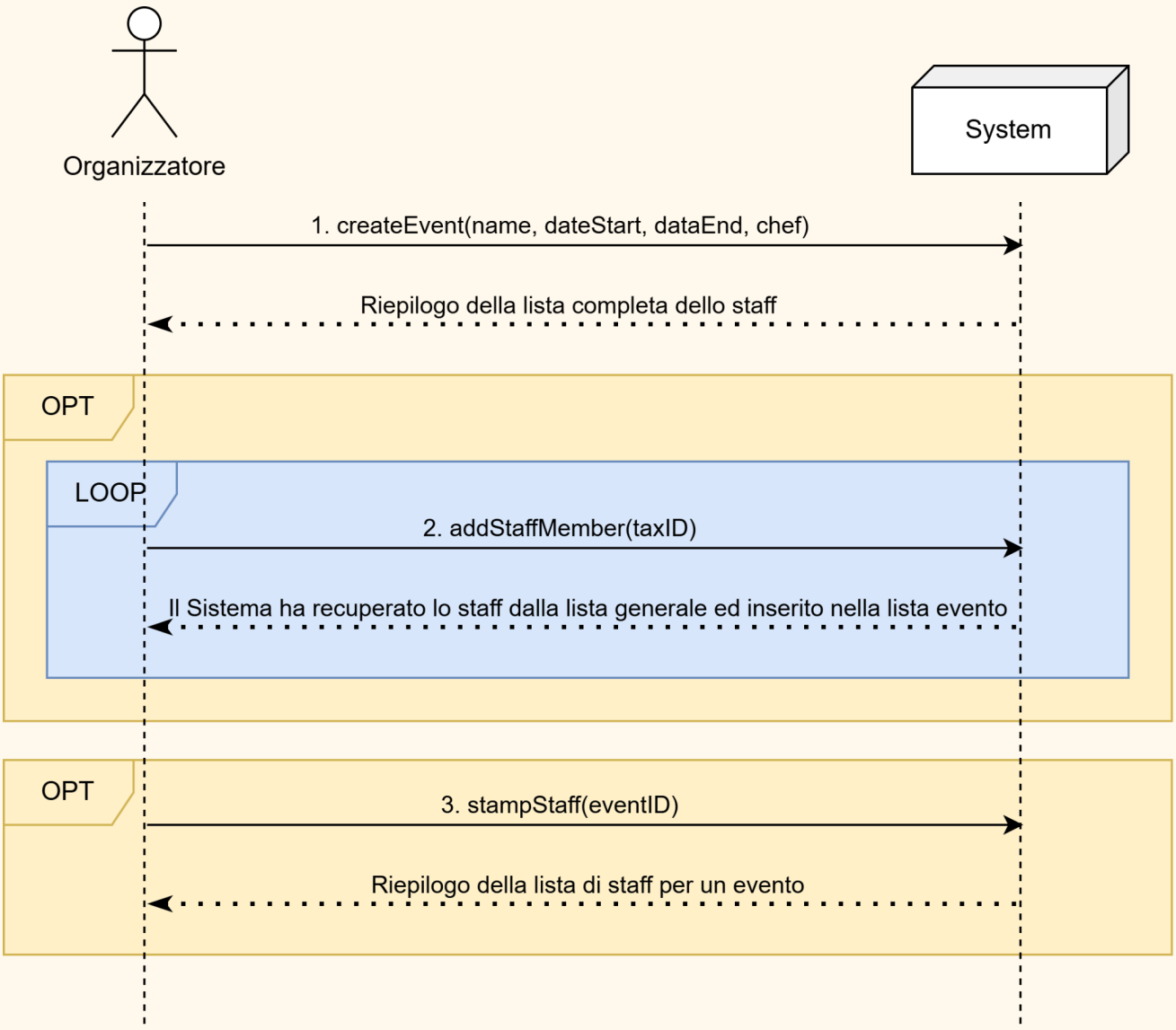
#	Attore	Sistema
1b	L'Organizzatore registra un nuovo membro inserendo dati completi.	Il sistema prova ad aggiungere un nuovo membro, ma il taxID è già presente nella lista, stampa un errore.
1c	L'Organizzatore rimuove definitivamente un membro dalla lista generale.	Il sistema prova a rimuovere il membro, ma è attualmente occupato, stampa un errore.
1c/1d	L'Organizzatore rimuove definitivamente un membro dalla lista generale. / L'Organizzatore inserisce il numero di ferie richieste da un membro del personale.	Il taxID fornito non corrisponde a nessuna entry della lista personale generale, stampa un errore.

Estensione 1a.1

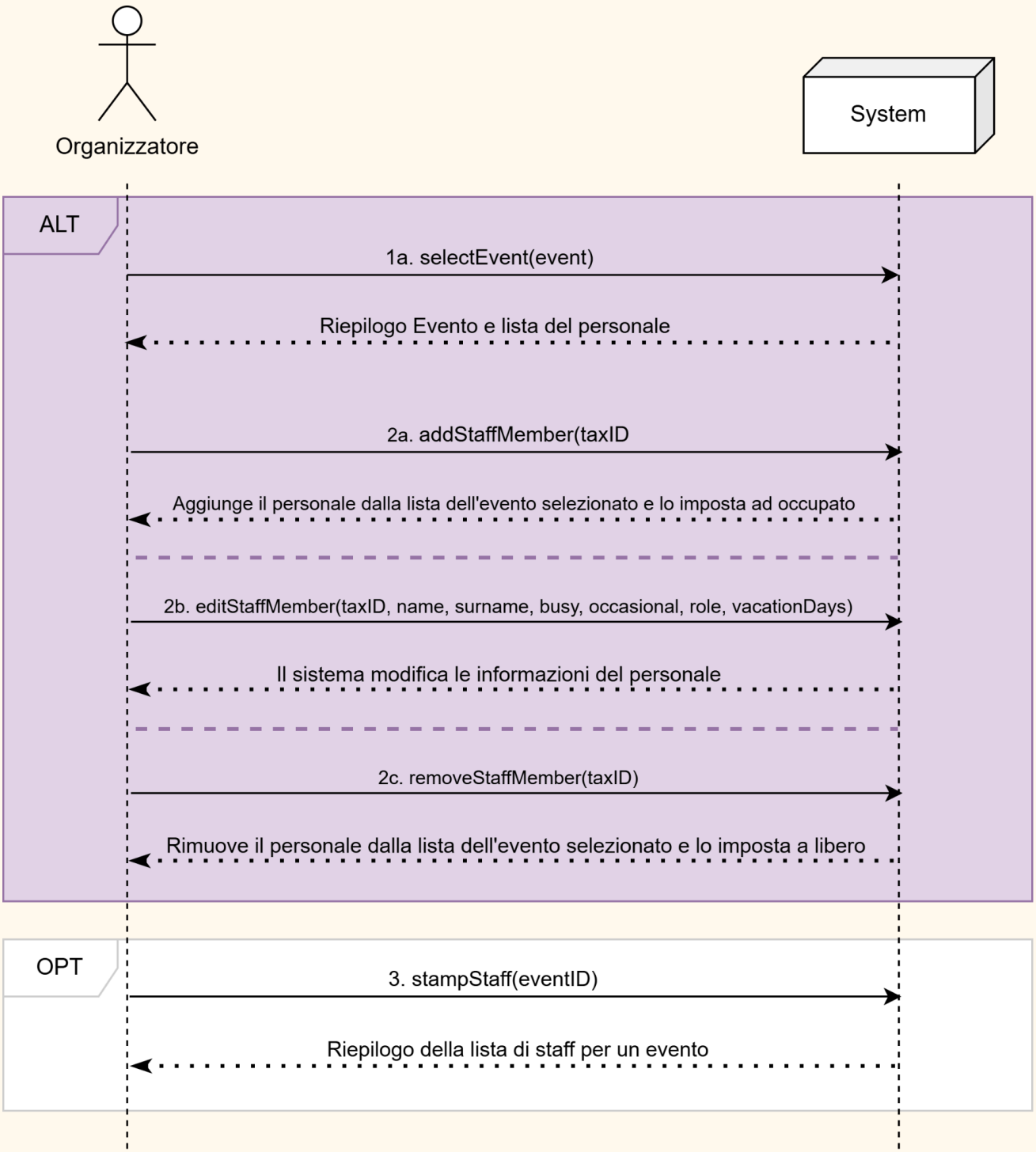
#	Attore	Sistema
1a.1	L'Organizzatore inserisce un servizio già creato all'evento selezionato.	Il sistema inserisce il servizio all'evento selezionato e calcola quanto staff è richiesto in base al tipo di servizio e se la location è a lunghe distanze si richiede un trasporto.


Personale Tot.
Nome
Cognome
Occupato
Ruolo
Occasionale
Ferie
Codice Fiscale
1
Visibile in
0...n
Evento
Titolo
N.Clienti
Tipo
Trasporto
1
Contiene
0...n
Personale per evento
Cofice Fiscale
Ruolo

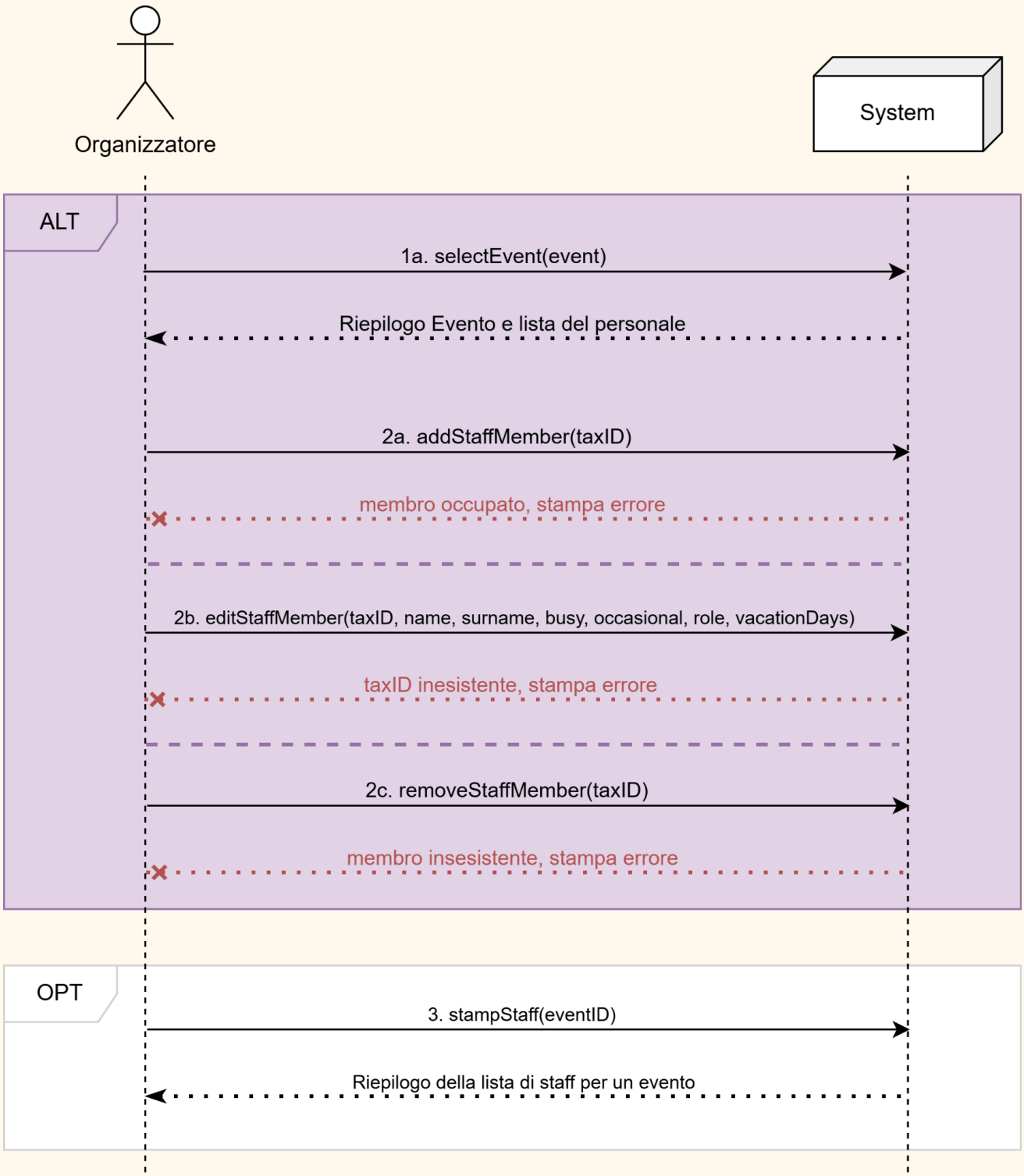
Diagrammi di sequenza di sistema per l'UC Catering



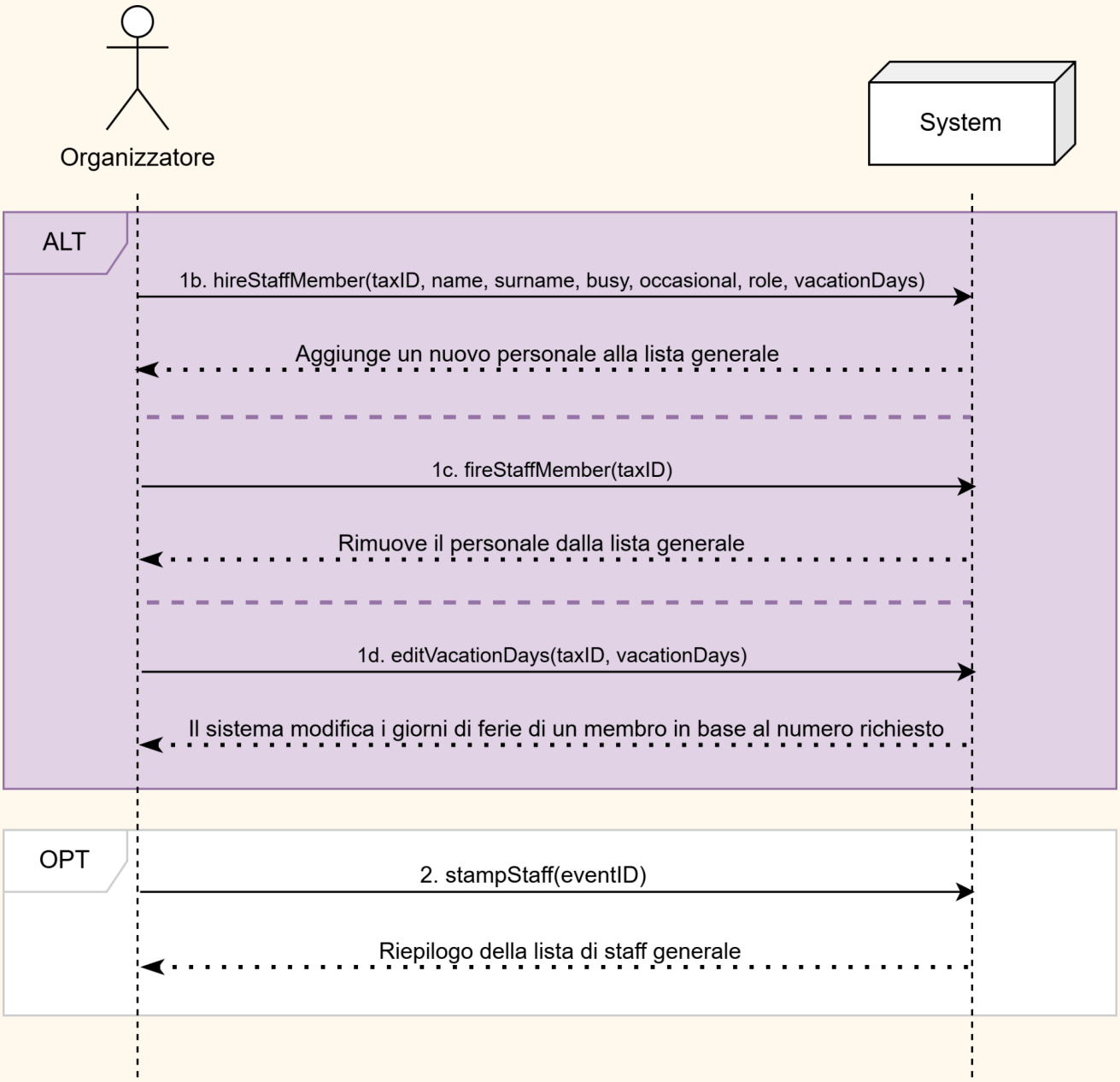
Estensione 1a/2a/2b/2c



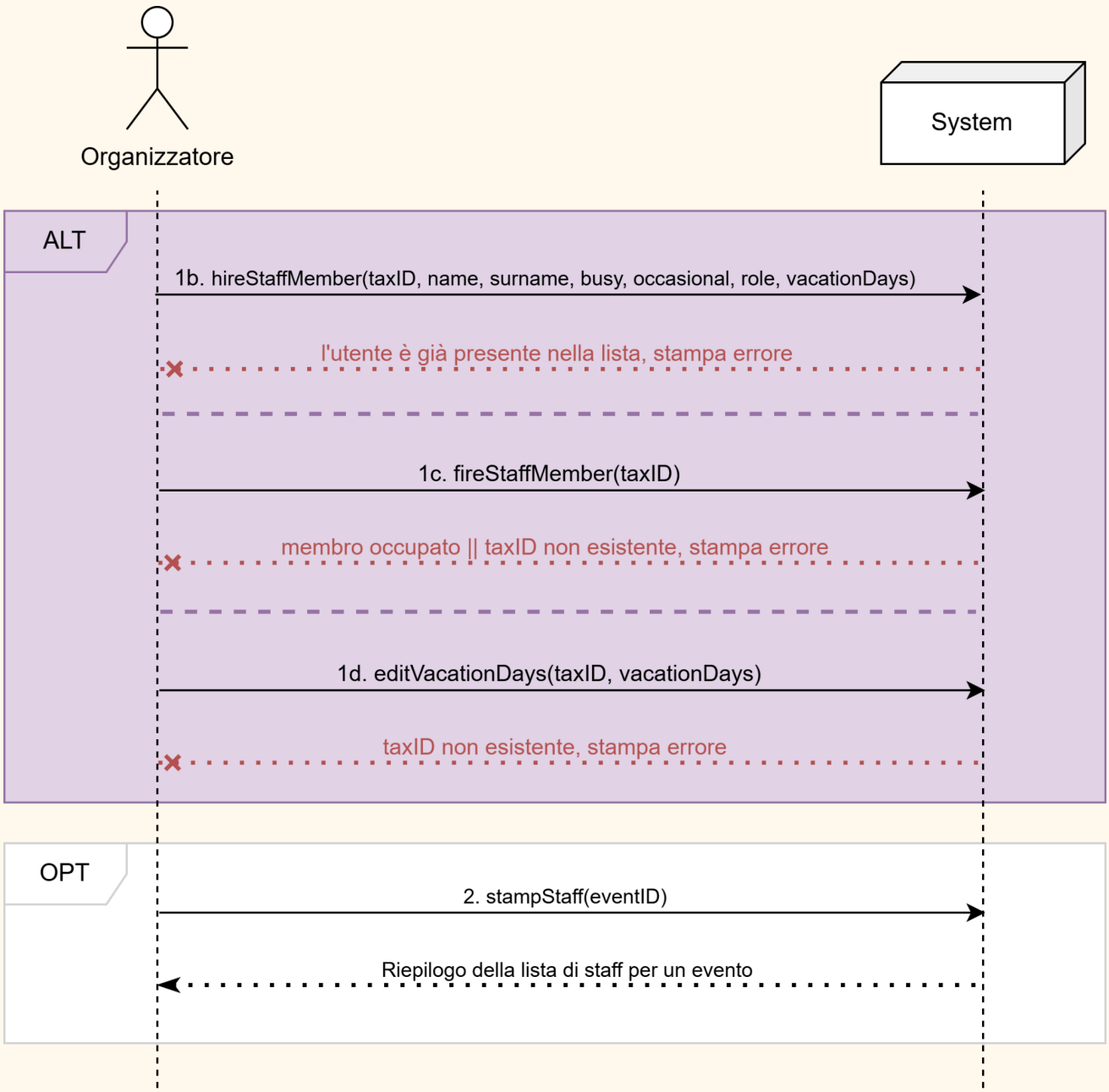
Eccezione 2a/2b/2c



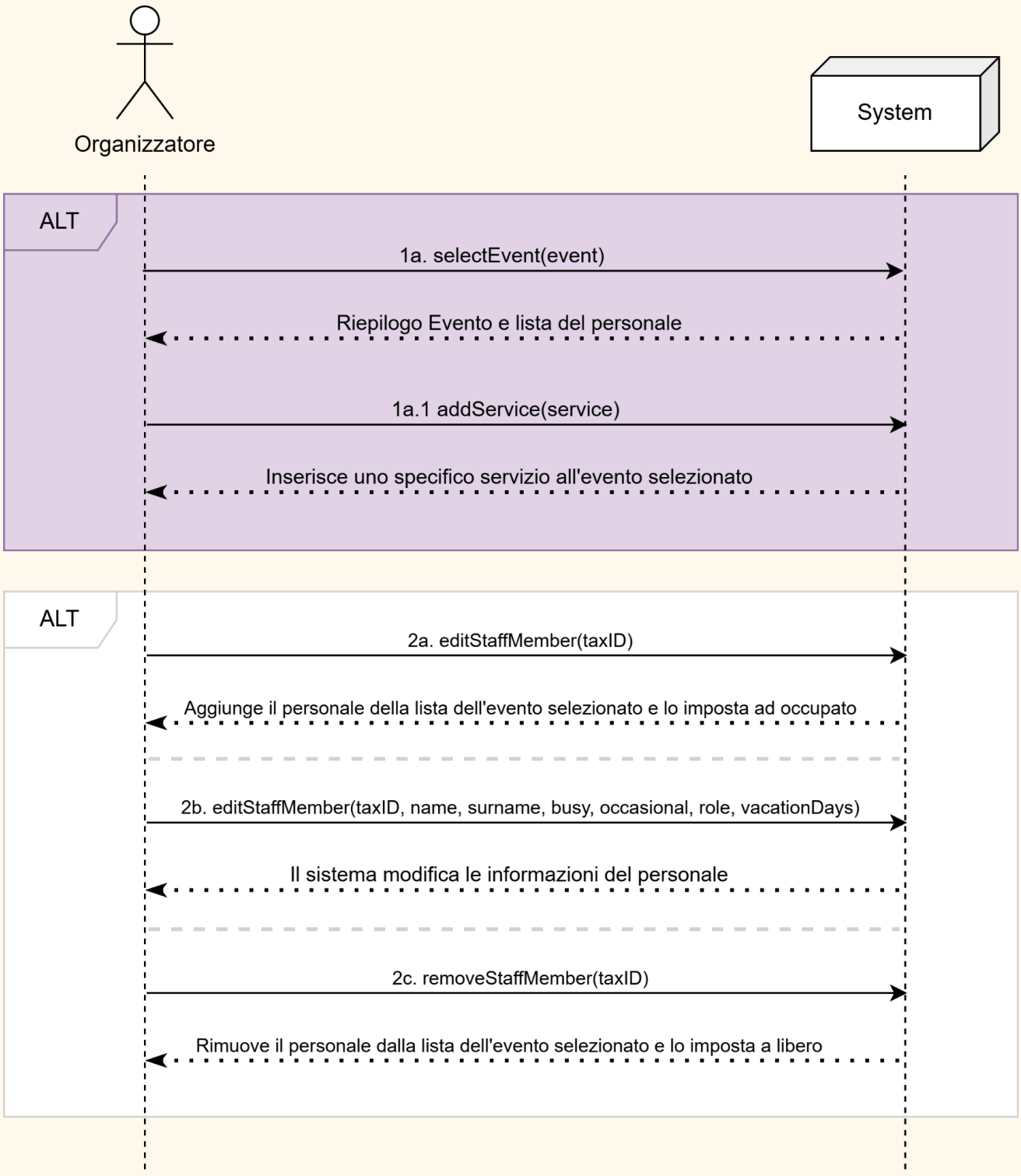
Estensione 1b/1c/1d



Eccezione 1b/1c/1d



Estensione 1a.1



CONTRATTI

pre-condizione generale: enum tipoEvento = {ECO, STANDARD, PREMIUM}
enum ruolo = {CUOCO, LAVAPIATTI, CAMERIERE, SOMMELIER,
MAÎTRE, BARTENDER, R-LOGISTICA}

1a: selectEvent(eventID)

pre-condizione:
- L'evento identificato da `eventID` deve esistere nel database del sistema.

post-condizione:
- Il sistema ha recuperato i dettagli dell'evento e la lista del personale attualmente associato ad esso.

1a.1: addService(service)

pre-condizione:
- Un `Evento` è attualmente selezionato nella sessione.
- Un `Service` il service passato come argomento deve essere valido

post-condizione:
- È stata formata un'associazione tra l'istanza di `Evento` selezionata e l'istanza `service`

1b: hireStaffMember(taxID, name, surname, busy, occasional, role, vacationDays)

pre-condizione:
- Il `taxID` fornito non deve essere già presente nell'anagrafica generale.

post-condizione:
- Il nuovo membro dello staff è aggiunto alla lista generale del catering.

1c: fireStaffMember(taxID)

pre-condizione:

- Esiste un membro dello staff con quel `taxID` nell'anagrafica generale.

post-condizione:

- Il membro viene rimosso definitivamente dal database del sistema.

1d: editVacationDays(taxID, vacationDays)

pre-condizione:

- Il membro dello staff deve esistere nel sistema.

post-condizione:

- Il valore del campo `vacationDays` del dipendente viene aggiornato secondo il numero richiesto dall'Organizzatore.

2a: addStaffMember(taxID)

pre-condizione:

- Un evento deve essere stato precedentemente selezionato; il membro dello staff identificato dal `taxID` deve esistere nell'anagrafica generale.

post-condizione:

- Il membro dello staff è aggiunto alla lista specifica dell'evento selezionato e il suo stato globale viene impostato su "occupato".

2b: editStaffMember(taxID, name, surname, busy, occasional, role, vacationDays)

pre-condizione:

- Esiste un'istanza `p` di `StaffMember` nel sistema tale che `p.taxID == taxID`.

post-condizione:

- L'attributo `p.name` è stato modificato in `name`.
- L'attributo `p.surname` è stato modificato in `surname`.
- L'attributo `p.busy` è stato modificato in `busy`.
- L'attributo `p.occasional` è stato modificato in `occasional`.
- L'attributo `p.role` è stato modificato in `role`.
- L'attributo `p.vacationDays` è stato modificato in `vacationDays`.

2c: removeStaffMember(taxID)

pre-condizione:

- Il membro dello staff con quel `taxID` deve essere presente nella lista dell'evento correntemente selezionato.

post-condizione:

- Il membro viene rimosso dalla lista dell'evento e il suo stato nell'anagrafica generale torna a "libero".

stampStaff(eventID)

pre-condizione:

- per `eventID` l'evento deve esistere.

post-condizione:

- Il sistema genera e restituisce un riepilogo leggibile dello staff richiesto.

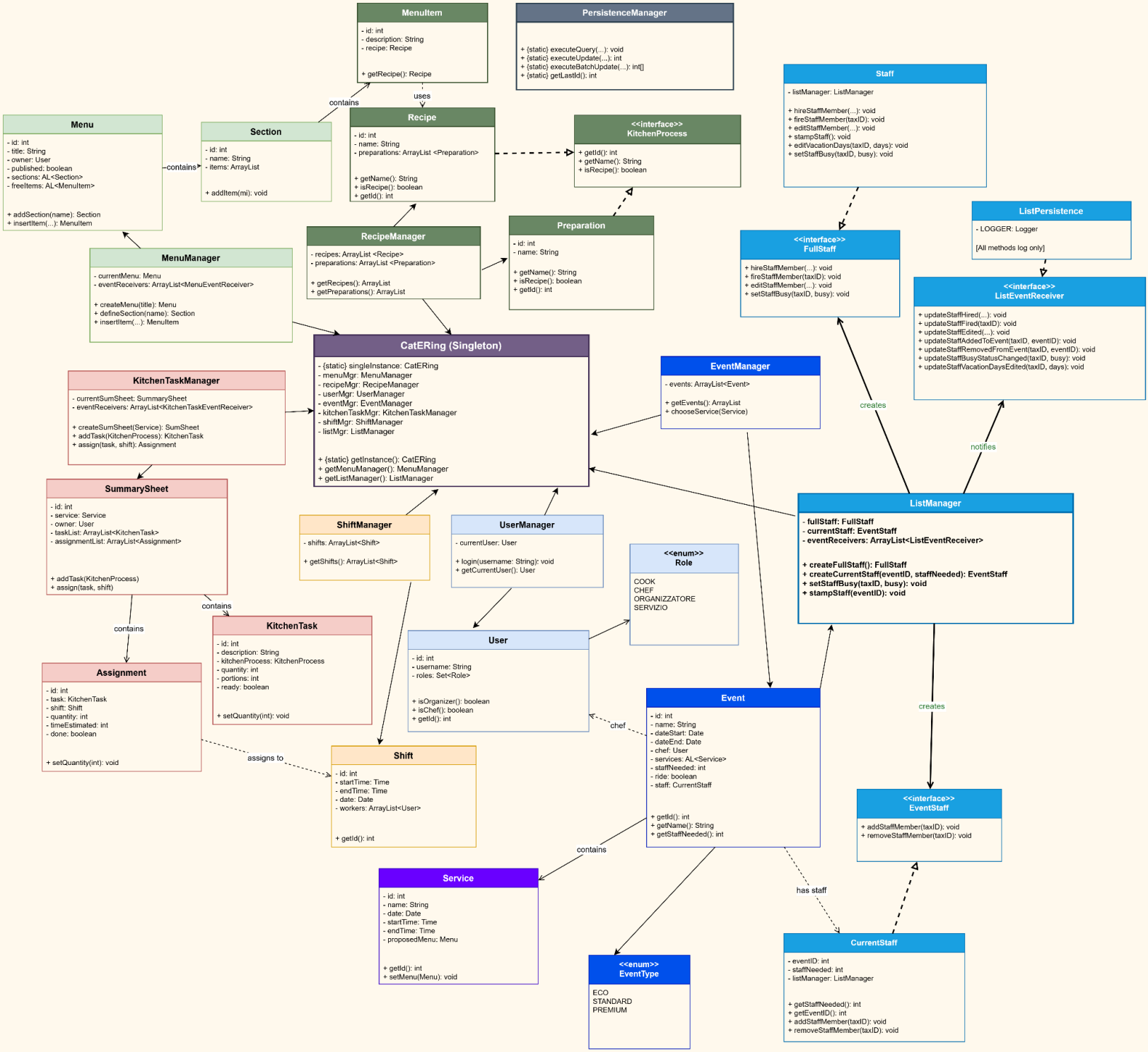
stampStaff()

pre-condizione: -

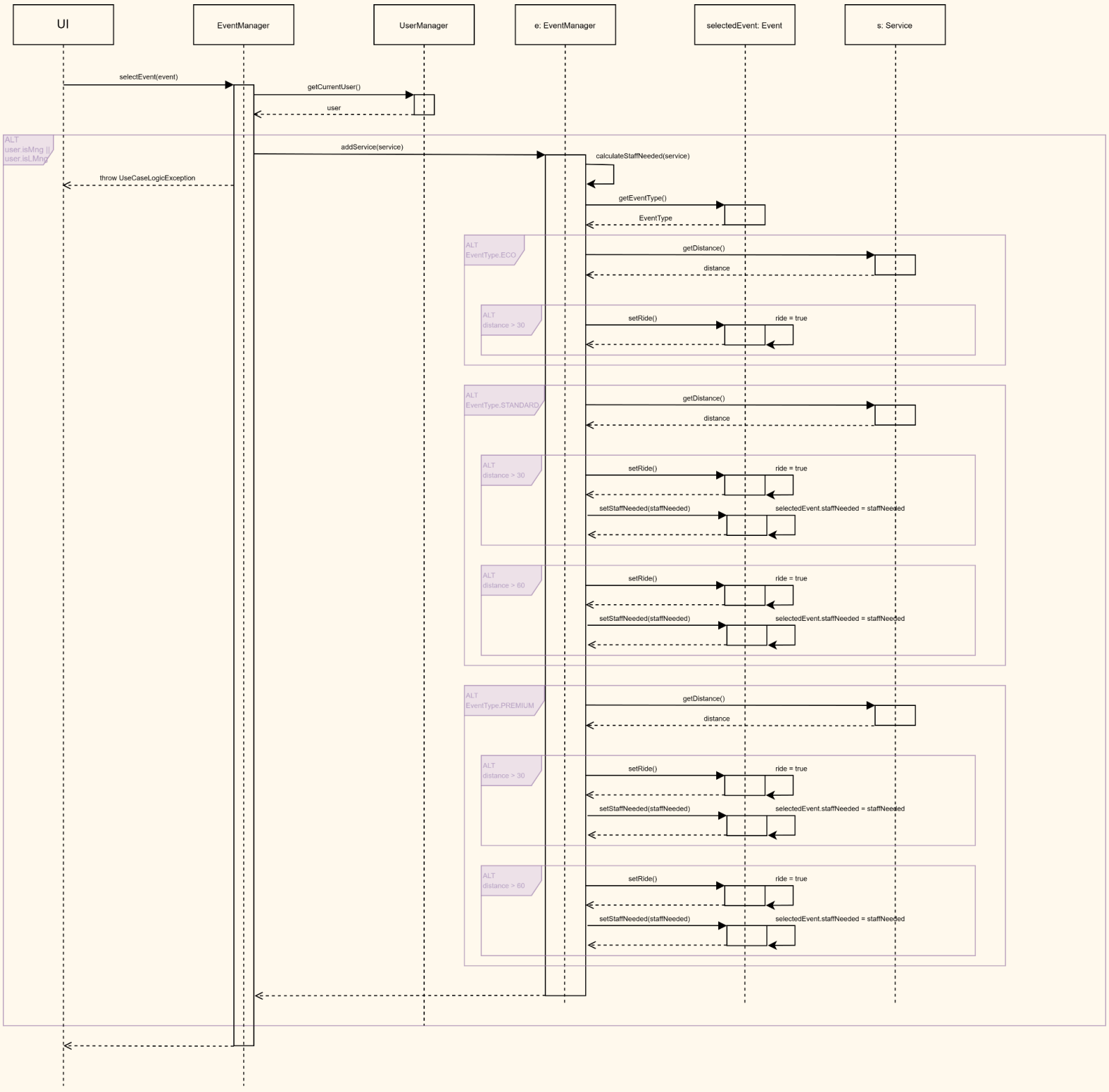
post-condizione:

- Il sistema genera e restituisce un riepilogo leggibile dello staff generale.

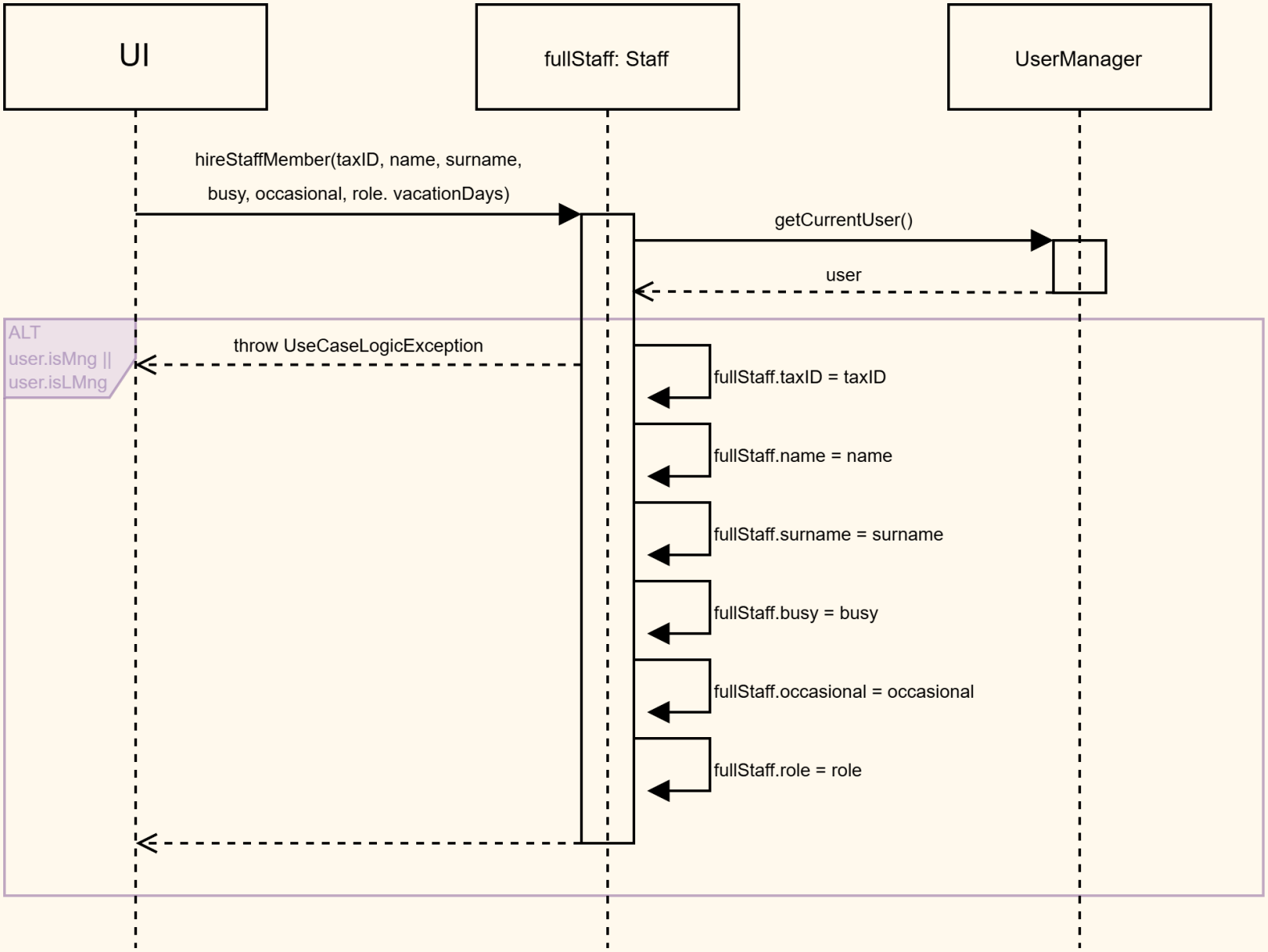
DCD


MenuItem
- id: int
- description: String
- recipe: Recipe
+ getRecipe(): Recipe
PersistenceManager
+ {static} executeQuery(...): void
+ {static} executeUpdate(...): int
+ {static} executeBatchUpdate(...): int[]
+ {static} getLastId(): int
Staff
- listManager: ListManager
+ hireStaffMember(...): void
+ fireStaffMember(taxID): void
+ editStaffMember(...): void
+ stampStaff(): void
+ editVacationDays(taxID, days): void
+ setStaffBusy(taxID, busy): void
contains
uses
Menu
- id: int
- title: String
- owner: User
- published: boolean
- sections: AL<Section>
- freeItems: AL<MenuItem>
+ addSection(name): Section
+ insertItem(...): MenuItem
contains
Section
- id: int
- name: String
- items: ArrayList
+ addItem(mi): void
Recipe
- id: int
- name: String
- preparations: ArrayList <Preparation>
+ getName(): String
+ isRecipe(): boolean
+ getId(): int
<<interface>>
KitchenProcess
+ getId(): int
+ getName(): String
+ isRecipe(): boolean
ListPersistence
- LOGGER: Logger
[All methods log only]
RecipeManager
- recipes: ArrayList <Recipe>
- preparations: ArrayList <Preparation>
+ getRecipes(): ArrayList
+ getPreparations(): ArrayList
Preparation
- id: int
- name: String
+ getName(): String
+ isRecipe(): boolean
+ getId(): int
<<interface>>
FullStaff
+ hireStaffMember(...): void
+ fireStaffMember(taxID): void
+ editStaffMember(...): void
+ setStaffBusy(taxID, busy): void
<<interface>>
ListEventReceiver
+ updateStaffHired(...): void
+ updateStaffFired(taxID): void
+ updateStaffEdited(...): void
+ updateStaffAddedToEvent(taxID, eventID): void
+ updateStaffRemovedFromEvent(taxID, eventID): void
+ updateStaffBusyStatusChanged(taxID, busy): void
+ updateStaffVacationDaysEdited(taxID, days): void
MenuManager
- currentMenu: Menu
- eventReceivers: ArrayList<MenuEventReceiver>
+ createMenu(title): Menu
+ defineSection(name): Section
+ insertItem(...): MenuItem
CatERing (Singleton)
- {static} singleInstance: CatERing
- menuMgr: MenuManager
- recipeMgr: RecipeManager
- userMgr: UserManager
- eventMgr: EventManager
- kitchenTaskMgr: KitchenTaskManager
- shiftMgr: ShiftManager
- listMgr: ListManager
+ {static} getInstance(): CatERing
+ getMenuManager(): MenuManager
+ getListManager(): ListManager
EventManager
- events: ArrayList<Event>
+ getEvents(): ArrayList
+ chooseService(Service)
creates
notifies
KitchenTaskManager
- currentSumSheet: SummarySheet
- eventReceivers: ArrayList<KitchenTaskEventReceiver>
+ createSumSheet(Service): SumSheet
+ addTask(KitchenProcess): KitchenTask
+ assign(task, shift): Assignment
SummarySheet
- id: int
- service: Service
- owner: User
- taskList: ArrayList<KitchenTask>
- assignmentList: ArrayList<Assignment>
+ addTask(KitchenProcess)
+ assign(task, shift)
ListManager
- fullStaff: FullStaff
- currentStaff: EventStaff
- eventReceivers: ArrayList<ListEventReceiver>
+ createFullStaff(): FullStaff
+ createCurrentStaff(eventID, staffNeeded): EventStaff
+ setStaffBusy(taxID, busy): void
+ stampStaff(eventID): void
ShiftManager
- shifts: ArrayList<Shift>
+ getShifts(): ArrayList<Shift>
UserManager
- currentUser: User
+ login(username: String): void
+ getCurrentUser(): User
<<enum>>
Role
COOK
CHEF
ORGANIZZATORE
SERVIZIO
contains
contains
KitchenTask
- id: int
- description: String
- kitchenProcess: KitchenProcess
- quantity: int
- portions: int
- ready: boolean
+ setQuantity(int): void
User
- id: int
- username: String
- roles: Set<Role>
+ isOrganizer(): boolean
+ isChef(): boolean
+ getId(): int
Assignment
- id: int
- task: KitchenTask
- shift: Shift
- quantity: int
- timeEstimated: int
- done: boolean
+ setQuantity(int): void
assigns to
Shift
- id: int
- startTime: Time
- endTime: Time
- date: Date
- workers: ArrayList<User>
+ getId(): int
chef
Event
- id: int
- name: String
- dateStart: Date
- dateEnd: Date
- chef: User
- services: AL<Service>
- staffNeeded: int
- ride: boolean
- staff: CurrentStaff
+ getId(): int
+ getName(): String
+ getStaffNeeded(): int
creates
<<interface>>
EventStaff
+ addStaffMember(taxID): void
+ removeStaffMember(taxID): void
contains
has staff
Service
- id: int
- name: String
- date: Date
- startTime: Time
- endTime: Time
- proposedMenu: Menu
+ getId(): int
+ setMenu(Menu): void
<<enum>>
EventType
ECO
STANDARD
PREMIUM
CurrentStaff
- eventID: int
- staffNeeded: int
- listManager: ListManager
+ getStaffNeeded(): int
+ getEventID(): int
+ addStaffMember(taxID): void
+ removeStaffMember(taxID): void

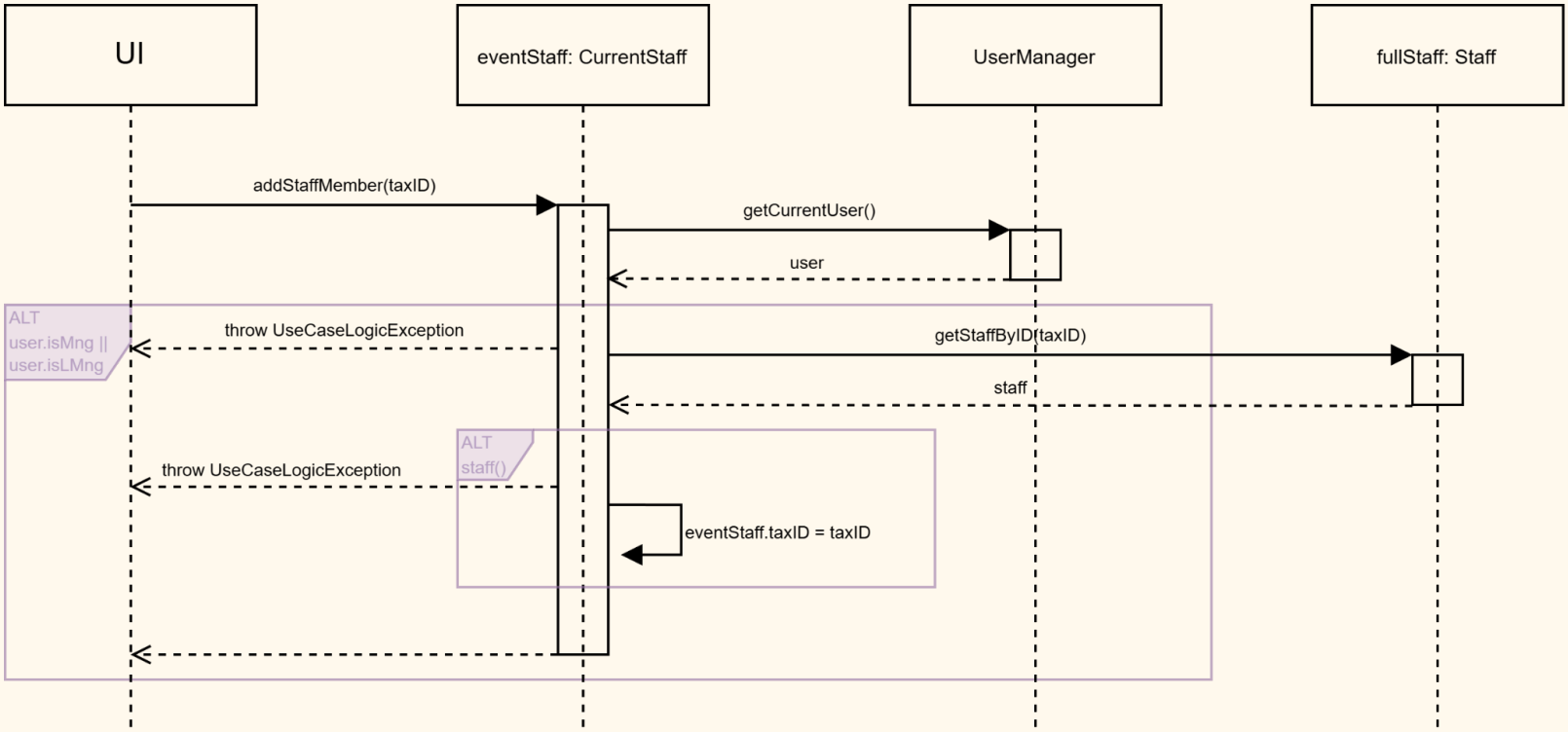
DSD (addService)



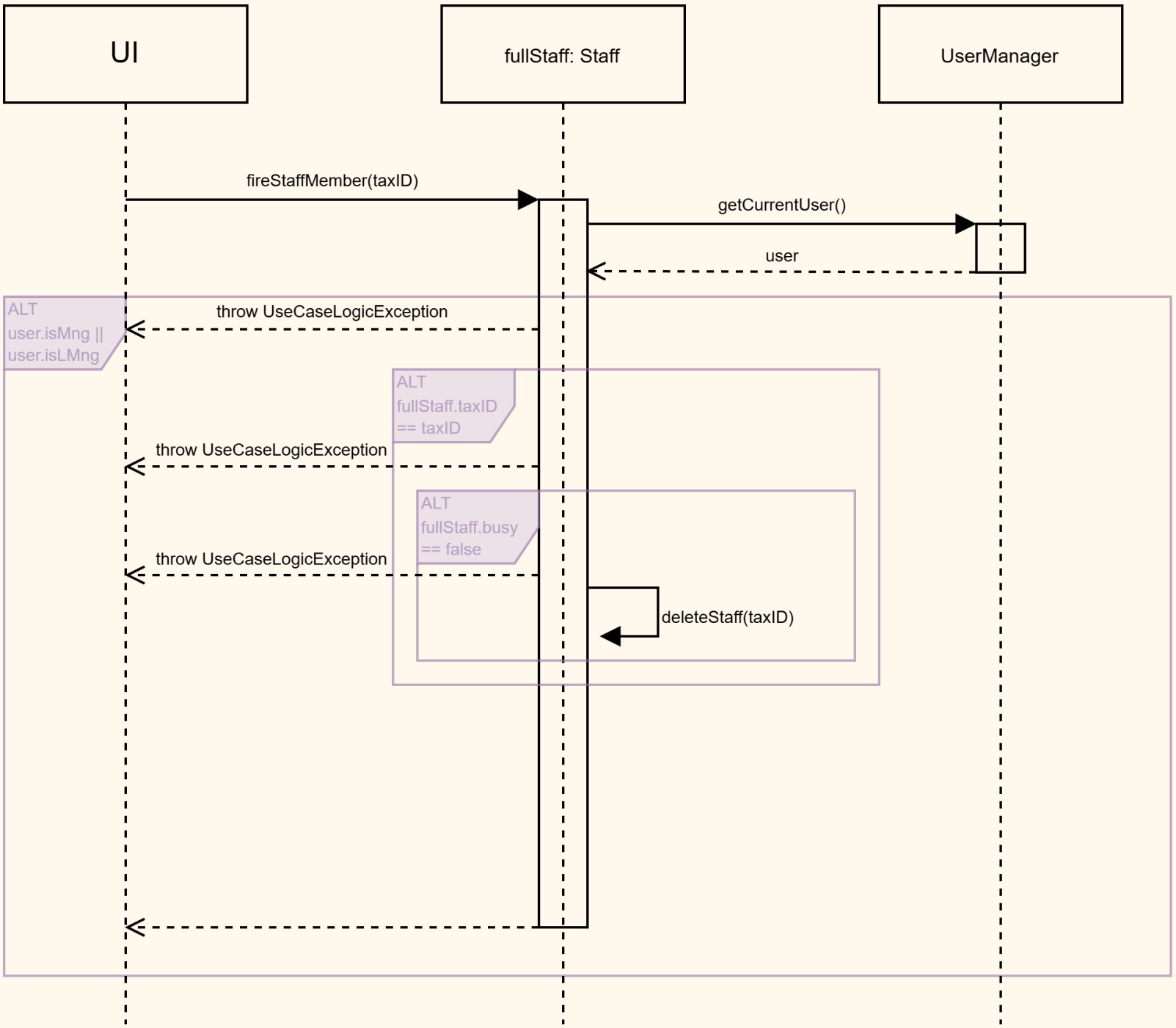
DSD (hireStaffMember)



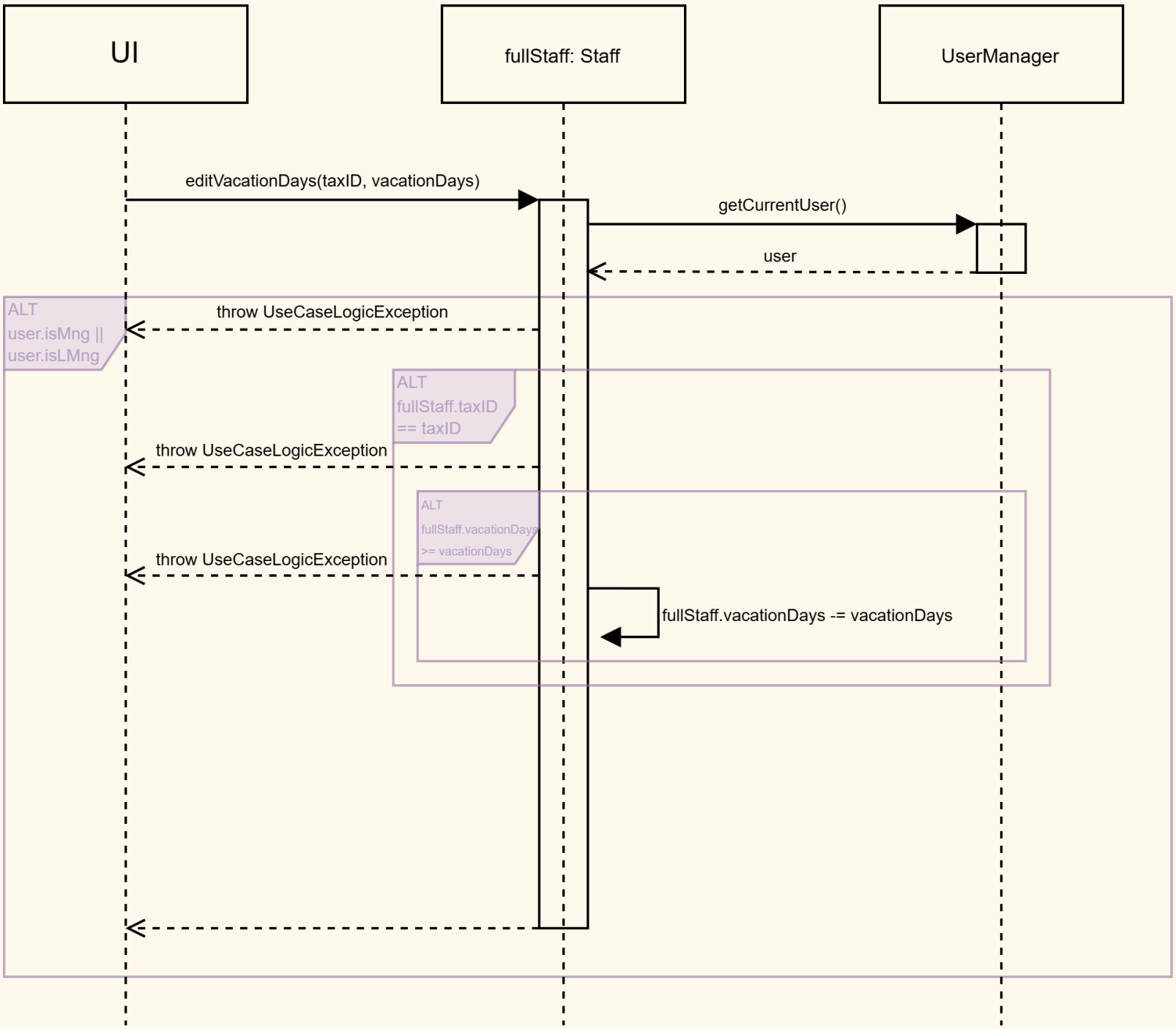
DSD (addStaffMember)



DSD (fireStaffMember)



DSD (editVacationDays)



DSD (createCurrentStaff)

